



University of
Sheffield

Exploration and Learning in the Lava Environment: SARSA and Q-Learning in Deterministic and Stochastic Settings

Vivek Vijay Choradia

Module Lead: Prof. Eleni Vasilaki

Module: COM3240 Reinforcement Learning

*A report submitted in fulfillment of the requirements
for the degree of BSc in Artificial Intelligence and Computer Science*

in the

Department of Computer Science

May 25, 2024

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by performing actions in an environment to maximize cumulative rewards. Unlike supervised learning, RL does not rely on labeled input/output pairs but instead uses the concept of exploration and exploitation to learn optimal behaviors through trial and error [1].

This report focuses on the implementation and analysis of RL algorithms within a grid world environment, here referred to as the “Lava” environment. The task involves a search-and-guide robot that navigates a grid to locate valuable artifacts while avoiding lava areas. The goal of the robot is to learn optimal paths to the artifacts, represented by high rewards, through interaction with the environment. The environment’s complexity increases progressively through three distinct scenarios:

- 1) **Single Target, Deterministic Rewards:** The environment has a single artifact with a fixed reward and deterministic transition dynamics.
- 2) **Dual Targets with Stochastic Rewards:** The environment contains two artifacts, each with rewards drawn from Gaussian distributions at the start of each episode, but with deterministic transition dynamics.
- 3) **Stochastic Transitions and Rewards:** Both the rewards and the transition dynamics are stochastic, representing the highest complexity.

To address these scenarios, we implement and evaluate two fundamental RL algorithms: SARSA (State-Action-Reward-State-Action) and Q-learning, along with exploration strategies such as ε -greedy and softmax policies. SARSA and Q-learning are widely used RL algorithms that update action-value functions to estimate the expected utility of actions in specific states, but they differ in their update rules. It is important to note that both algorithms rely on the Markovian property, where the future state depends only on the current state and action, not on the sequence of events that preceded it.

A. SARSA

SARSA (State-Action-Reward-State-Action) is an on-policy RL algorithm. It updates the action-value function based on the action actually taken by the agent, following its current policy. The update rule for SARSA is given by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (1)$$

where:

- $Q(s_t, a_t)$ is the action-value function for state s_t and action a_t ,
- α is the learning rate,
- r_{t+1} is the reward received after taking action a_t ,
- γ is the discount factor,
- s_{t+1} and a_{t+1} are the next state and action, respectively.

B. Q-Learning

Q-Learning is an off-policy RL algorithm. It updates the action-value function based on the maximum possible reward of the next state, regardless of the action actually taken. The update rule for Q-Learning is given by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (2)$$

where the terms are defined similarly to those in SARSA, but the update uses the maximum action-value function $\max_a Q(s_{t+1}, a)$.

These algorithms were implemented in an OOP format (refer to Appendix B for implementation code snippets). The subsequent sections detail the experimental setup, results, and analysis of each task, providing insights into the challenges and solutions associated with deterministic and stochastic environments in reinforcement learning.

I. SINGLE TARGET, DETERMINISTIC REWARDS

This section explores the simplest scenario in the grid world environment: a single target with deterministic rewards. The environment consists of a grid where the robot can move in four possible directions: North, South, East, and West. The robot’s goal is to navigate from a fixed starting position to the target location, which provides a high reward. The environment’s transition dynamics are deterministic, meaning the robot’s actions lead to predictable and consistent outcomes.

To control the randomness challenge in RL, a file of 100 seeds was created and saved to use throughout the experiments. Using these seeds ensures that the results are reproducible and comparable across different runs and settings.

A. Action Policy: ε -greedy

The ε -greedy policy is an exploration strategy commonly used in RL. This policy balances exploration and exploitation by choosing random actions with a small probability ε and exploiting the best-known

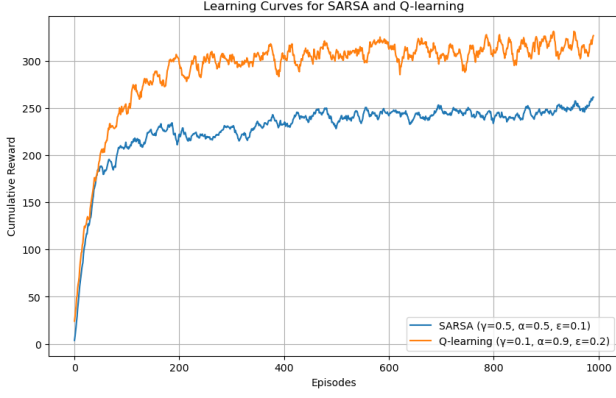


Fig. 1. Learning Curves for SARSA and Q-learning with Specific Hyperparameters

actions the rest of the time. It can be defined as follows:

- With probability $1 - \varepsilon$, choose the action a that maximises the action-value function $Q(s, a)$.
- With probability ε , choose a random action a' from the action space.

This approach ensures the agent explores sufficiently while exploiting known information to maximize rewards.

B. Results

The learning curve for the SARSA and Q-learning algorithms with specific hyperparameters is generated by plotting the cumulative reward collected per episode against the number of episodes. This helps in understanding the agent's learning progress over time.

For this scenario, the following hyperparameters were chosen for each algorithm which were run over 1000 episodes:

- **SARSA:**
 - Learning Rate (α): 0.5
 - Discount Factor (γ): 0.5
 - Exploration Rate (ε): 0.1
- **Q-learning:**
 - Learning Rate (α): 0.9
 - Discount Factor (γ): 0.1
 - Exploration Rate (ε): 0.2

The chosen hyperparameters were based on multiple runs to optimize the agent's performance. Refer to Appendix C.

With these hyperparameters, the preferred movement directions are shown in Figure 2 and 3.

C. Exploration of Hyperparameters

In RL, hyperparameters critically influence algorithm performance. For our grid world environment, we varied three key hyperparameters: learning rate (α),

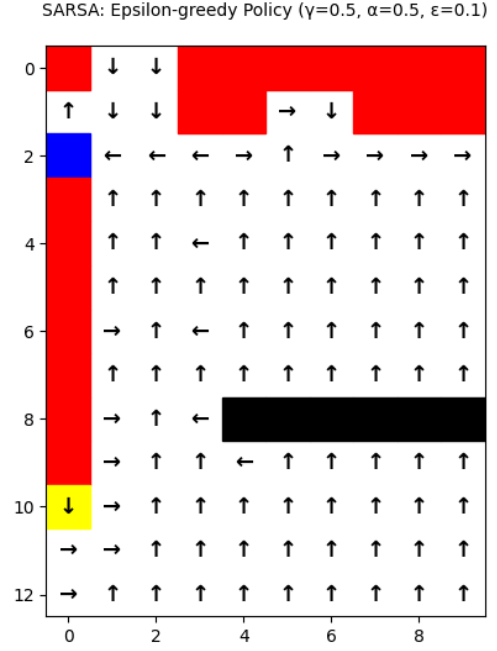


Fig. 2. Preferred Movement Direction

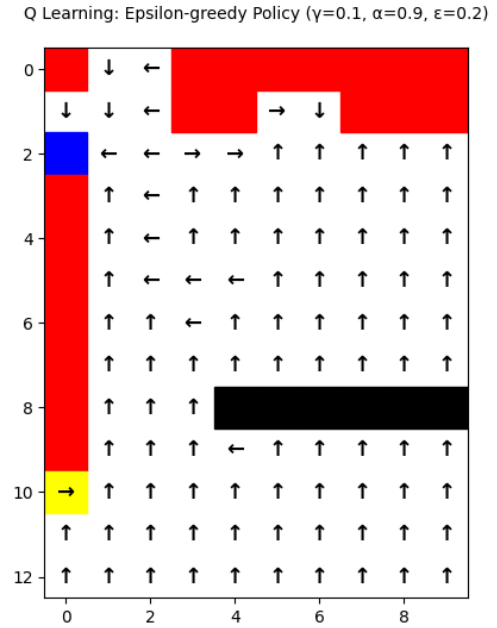


Fig. 3. Preferred Movement Direction

discount factor (γ), and exploration rate (ε).

1) **Learning Rate (α):** The learning rate dictates how new information updates old knowledge. A higher α accelerates learning but can cause instability, while a lower α ensures stable but slower learning. SARSA performed best with $\alpha = 0.5$, whereas Q-learning required a higher $\alpha = 0.9$ for comparable performance in deterministic settings.

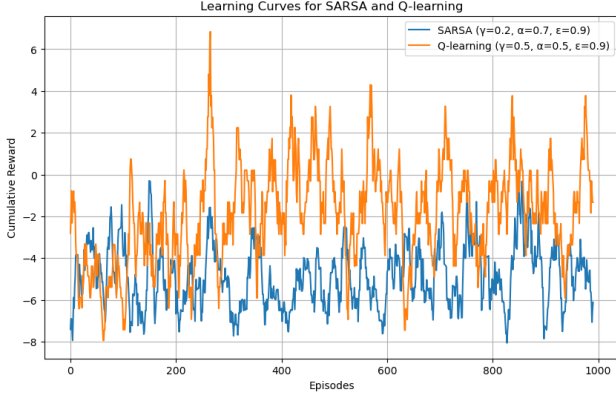


Fig. 4. Bad learning curve due to inadequate hyperparamters

2) **Discount Factor (γ)**: The discount factor balances the importance of future versus immediate rewards. A higher γ makes the agent value future rewards more, crucial for environments with delayed rewards. For SARSA, $\gamma = 0.5$ provided an effective balance across 1000 episodes. Q-learning, with $\gamma = 0.1$, performed robustly, favoring immediate rewards as predictors of optimal policy.

3) **Exploration Rate (ϵ)**: The exploration rate controls the balance between exploring new actions and exploiting known rewards. Higher ϵ values encourage exploration but may miss optimal actions. With $\epsilon = 0.1$ for SARSA and $\epsilon = 0.2$ for Q-learning, there was sufficient exploration while exploiting best-known actions to maximize rewards.

Figure 1 and Figure 4 illustrate how these parameters impact learning. In settings with poorly chosen hyperparameters, as shown in the second figure, the algorithms struggle to converge, demonstrating the importance of tuning. For example, with SARSA ($\gamma = 0.2$, $\alpha = 0.7$, $\epsilon = 0.9$), the agent's performance was significantly erratic, reflecting excessive exploration and inadequate exploitation. Similarly, Q-learning with a higher γ of 0.5 and a high ϵ of 0.9 showed frequent spikes in performance, indicating instability in policy development.

D. ϵ -greedy vs Softmax

This subsection compares the ϵ -greedy policy with the Softmax policy to determine which strategy yields better rewards.

Softmax

The Softmax policy assigns probabilities to actions based on their relative Q-values, promoting actions with higher values but still allowing for exploration.

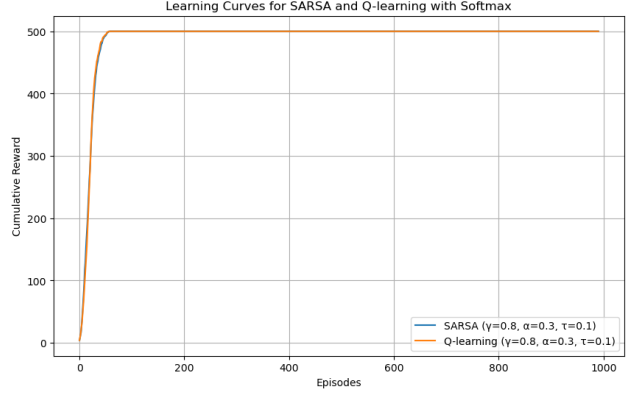


Fig. 5. Learning Curve with Softmax

The Softmax function is defined as follows:

$$\pi(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

where:

- $\pi(a|s)$ is the probability of taking action a in state s .
- $Q(s, a)$ is the Q-value of action a in state s .
- τ is the temperature parameter, controlling the degree of exploration.

When experimenting RL algorithms with softmax policy (implementation shown in Appendix D, we got to know that using softmax policy is giving up way more cumulative rewards than ϵ -greedy policy. With using softmax, with hyperparamters $\gamma=0.8$, $\alpha=0.3$, $\tau=0.1$. The learning curve can be shown in Figure 5. And therefore, softmax policy will be used for subsequent tasks.

II. DUAL TARGETS WITH STOCHASTIC REWARDS

In this section, the grid world environment is extended to include two target locations, each associated with stochastic rewards. The stochastic nature of the rewards introduces variability, increasing the complexity of the learning task. The agent must identify and consistently reach the target with the higher expected reward. Following observations that the Softmax policy yielded superior results in the single target, deterministic rewards scenario, all subsequent experiments will exclusively use the Softmax policy.

Implementing SARSA and Q-learning agents with the optimal hyperparameters from the previous environment, the following results were obtained as shown in Figure 6. Agent reaches to a stability after certain number of episodes in Environment 1 but it never reaches the point of stability for environment 2. The color filled in the graph is showing the

variance for all the 4 agents. It can be visualised that for environment 2, the agents deviates quite a lot from its path unlike for environment 1, this is because they keep of getting confused between the two goal states.

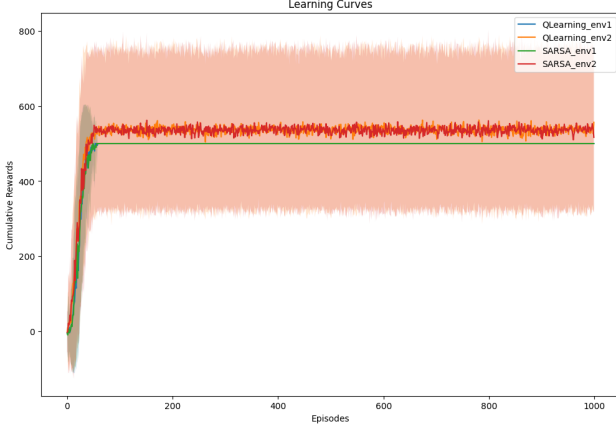


Fig. 6. Environment 1 vs Environment 2 with the same algorithm

Upon close examination, in Environment 1, the algorithm provides policy stability which is not observed in Environment 2. This instability is attributed to stochastic rewards. Stochastic rewards introduce variability, which impacts the agent’s ability to consistently identify the optimal policy.

To further investigate, the total number of steps to reach the goal over episodes was plotted (refer to Figure E). This analysis was conducted to understand the learning performance and efficiency of the agents in navigating the environment. It revealed that in Environment 1, SARSA and Q-learning take 43 and 57 episodes, respectively, to reach stability, whereas in Environment 2, it took 78 episodes for SARSA and 68 for Q-learning. This particular analysis was conducted with a single seed (68) instead of the usual 100 seeds to highlight specific instances of variability and their impact on learning stability. The findings indicate that stochastic rewards significantly affect the learning process, making it more challenging for the agent to stabilize its policy.

A. Parameter Optimization: Introducing τ -decay

τ -decay is a technique where the parameter τ , used in the Softmax policy for action selection, is gradually decreased over time. Mathematically, τ -decay can be expressed as:

$$\tau = \max(\tau_{\min}, \tau \times \text{decay rate})$$

where decay_rate is a value slightly less than 1. This method improves the algorithm by initially promot-

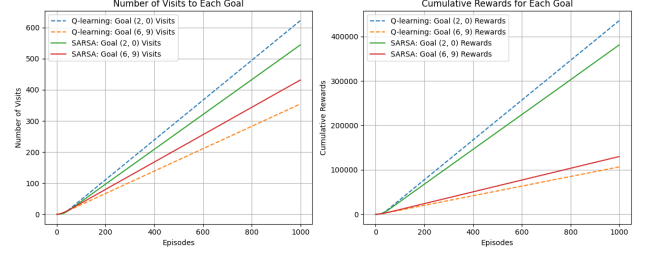


Fig. 7. SARSA and Q Learning Agent with Tau-Decay

ing exploration and progressively favoring exploitation as the agent learns more about the environment.

In the implementation, applying τ -decay resulted in improved policy stability and higher cumulative rewards in Environment 2, as demonstrated by a smoother learning curve and more consistent performance across episodes.

B. Challenge: Balancing Two Goal States

Balancing two goal states with stochastic rewards requires the agent to adapt to the variability and consistently identify the target with the higher expected reward. This challenge necessitates robust learning algorithms capable of handling uncertainty. To understand this challenge itself, Figure 7 was plotted in such a way that we can compare between the two goals. It can be seen that the agent visits the goal with a higher reward than

C. Extending Current Algorithm with Eligibility Traces (λ)

Eligibility traces are a mechanism that combines the benefits of Monte Carlo methods and dynamic programming. They allow for updates to not only the current state-action pair but also to previous state-action pairs, decayed by a factor λ . The updated rule for eligibility traces is:

$$E(s, a) = \begin{cases} E(s, a) + 1 & \text{if } (s, a) = (s_t, a_t) \\ \gamma \lambda E(s, a) & \text{otherwise} \end{cases}$$

This approach enhances the existing algorithm by providing a more comprehensive update mechanism, which is particularly useful in environments with delayed rewards or stochastic variability.

Applying eligibility traces to the current algorithm helps address the challenge of balancing two goal states by more effectively propagating reward information through the state-action space. This leads to faster convergence and more stable policies, particularly when hyperparameters such as λ and τ -decay are carefully tuned. Further research involved experimenting with different λ -trace values (0.0, 0.3,

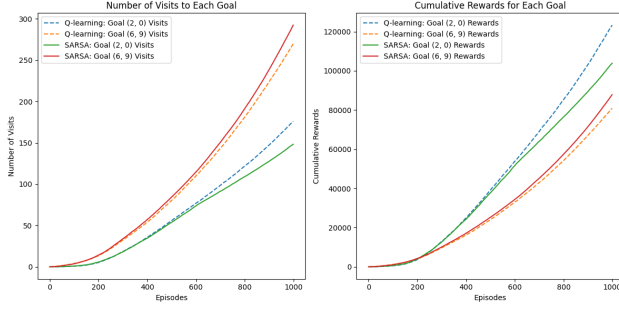


Fig. 8. Performance of SARSA and Q Learning with eligibility traces in task 3 environment.

0.5, 0.7, 0.9, 1.0), revealing that higher λ values resulted in poorer performance.

III. STOCHASTIC TRANSITIONS AND REWARDS

Task 3 introduces increased complexity in the reinforcement learning environment by incorporating stochastic transitions alongside stochastic rewards. This setting aims to closely mimic real-world scenarios, where actions do not always yield predictable outcomes due to environmental uncertainties. In this task, a robot navigating through a grid world faces both variable rewards at target locations and unpredictable movements, adding layer of challenge to the learning process.

The environment consists of two reward locations, each offering a stochastic reward drawn from a Gaussian distribution at the start of each episode. The primary objective for the robot remains to locate and consistently reach the target with the highest expected reward. Unlike previous tasks, the robot’s movements are not always deterministic. For instance, an intended move north might result in the robot staying in the same position due to a “slip”, simulating real-world imperfections and uncertainties in control systems.

On experimenting with the modified algorithm from Task 2, it was found that the performance was comparatively worse. Although the algorithm could identify which goal state provided higher rewards, it only visited the better goal state between 250-300 times out of 1000 episodes, compared to 550-650 times in the Task 2 environment. This pattern was also reflected in the cumulative rewards. The performance degradation can be visually seen in Figure 8.

A. Hyperparameter Optimization

Multiple experiments were conducted to optimize the hyperparameters, but none of them yielded better results than the previous settings. In future work,

more sophisticated experimentation and advanced techniques will be employed to achieve improved performance.

B. Challenged poised by Stochastic Transitions and Rewards

The addition of stochastic transitions and stochastic reward values presents significant challenges in reinforcement learning. Stochastic transitions imply that the outcome of an agent’s action is not deterministic, introducing a level of unpredictability that complicates the learning process. The agent can no longer rely on a specific action leading to a predictable result, which necessitates more sophisticated exploration strategies. This increased uncertainty can lead to slower convergence, as the agent needs to gather more data to accurately model the environment’s dynamics and optimize its policy. Balancing exploration and exploitation becomes more complex, requiring the agent to explore a wider range of actions to understand the probabilistic nature of the transitions fully.

Stochastic reward values further exacerbate the learning challenge by introducing variability in the feedback the agent receives. This variability can obscure the true value of actions and states, making it harder for the agent to distinguish between noise and meaningful signals. As a result, the agent may require more episodes to accurately estimate the expected rewards and make informed decisions. The combination of stochastic transitions and rewards can lead to delayed learning and suboptimal performance, as the agent must continuously adapt to a fluctuating environment. This is reflected in the slow increase in cumulative rewards seen in the right graph of Figure 8, indicating the prolonged period required for the algorithms to adapt to the stochastic dynamics. These challenges necessitate robust learning algorithms that can handle noise and uncertainty effectively, ensuring stable and reliable learning outcomes in complex and unpredictable settings.

C. Algorithm Modification: Introducing Simple Neural Network

To address the challenges imposed by stochastic transitions and reward values, an Artificial Neural Network (ANN) was introduced. The neural network employed the Rectified Linear Unit (ReLU) activation function to enhance the agent’s ability to learn and generalize in the stochastic environment. Despite the potential advantages of using ANNs, the initial implementation did not yield significant

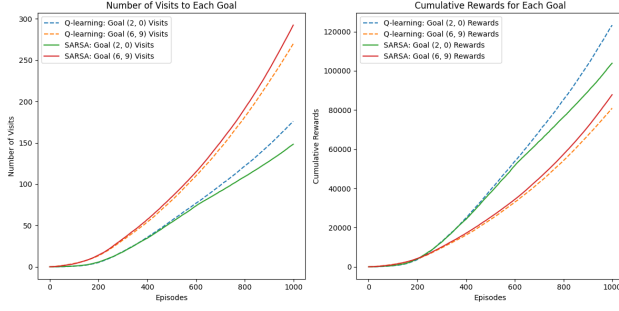


Fig. 9. SARSA and Q learning agent with ANN

improvements due to insufficient testing and tuning of hyperparameters (this can be seen in Figure 9).

D. Artificial Neural Network with ReLU

An Artificial Neural Network (ANN) consists of multiple layers of interconnected neurons, where each neuron applies a nonlinear activation function to its input. The ReLU activation function is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

This function introduces nonlinearity into the model while maintaining computational efficiency, as it allows for faster training compared to other activation functions like sigmoid or tanh. The use of ReLU helps in mitigating the vanishing gradient problem, which can occur with sigmoid or tanh activations during backpropagation in deep networks.

The neural network architecture included an input layer corresponding to the state representation, one or more hidden layers with ReLU activations, and an output layer producing Q-values for each possible action. The weights of the network were updated using the following gradient descent update rule:

$$\Delta W = \alpha \cdot \delta \cdot h$$

where:

- ΔW represents the change in weights,
- α is the learning rate,
- δ is the error term,
- h is the activation from the previous layer.

For the update of the bias term:

$$\Delta b = \alpha \cdot \delta$$

where b is the bias and δ is the error term derived from the difference between the predicted and target Q-values.

E. ANN with ReLU, Gradient Descent, and Backpropagation

To further enhance the performance of the neural network, an advanced version incorporating gradient descent and backpropagation was developed. Backpropagation is a method used to calculate the gradient of the loss function with respect to each weight by applying the chain rule, thereby efficiently updating the weights in the network. The backpropagation algorithm follows these steps:

- 1) **Forward Pass:** Calculate the output of the network by propagating the input through each layer.
- 2) **Compute Loss:** Calculate the loss function, typically the mean squared error (MSE):

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where y_i is the target value, $\hat{y}_i$ is the predicted value, and N is the number of training samples.

- 3) **Backward Pass:** Calculate the gradient of the loss function with respect to each weight by propagating the error backward through the network.
- 4) **Weight Update:** Update the weights using the gradient descent update rule mentioned earlier.

However, the implementation of this advanced neural network faced computational efficiency issues. The computational resources required to train the network with gradient descent and backpropagation were significant, leading to extended training times and difficulties in meeting the project's deadlines. Consequently, while the ANN with ReLU activation showed promise, the lack of extensive hyperparameter tuning and the computational demands of the full implementation with gradient descent and backpropagation prevented it from delivering optimal results within the available timeframe.

These challenges highlight the need for further experimentation and optimization of neural network architectures and training methodologies to effectively harness their potential in stochastic reinforcement learning environments.

REFERENCES

- [1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

APPENDIX A REPRODUCIBILITY

To reproduce the results shown in the report, open Assignment.ipynb file and click on “Run all”. This will allow you to produce all the results shown in this report. It has to be made sure that all the supporting files are in the same directory.

APPENDIX B IMPLEMENTING BASE RL AGENTS

RL agents were implemented in an OOP structure to reusability of the code.

```
# Base class for RL agents
class RLagent:
    def __init__(self, env: GridWorld, policy_type='softmax', alpha=0.1, gamma=0.99, episodes=1000):
        self.env = env
        self.policy_type = policy_type
        self.alpha = alpha
        self.gamma = gamma
        self.episodes = episodes

    # Softmax function to compute probabilities
    def softmax(self, q_values, tau=1.0):
        exp_q = np.exp(q_values / tau - np.max(q_values / tau))
        return exp_q / np.sum(exp_q)

    # Function to select actions based on policy type
    def select_action(self, q_table, state, epsilon=0.1, tau=1.0):
        if self.policy_type == 'epsilon-greedy':
            if np.random.rand() < epsilon:
                return np.random.randint(len(q_table[state]))
            else:
                return np.argmax(q_table[state])
        elif self.policy_type == 'softmax':
            action_probabilities = self.softmax(q_table[state], tau)
            return np.random.choice(np.arange(len(q_table[state])), p=action_probabilities)
        else:
            raise ValueError("Unsupported policy type")
```

Fig. 10. Base RL Agent


```

# Q-learning agent
class QLearningAgent(RLagent):

    def __init__(self, env, policy_type='softmax', alpha=0.1, gamma=0.99, episodes=1000):
        super().__init__(env, policy_type, alpha, gamma, episodes)
        self.q_table = np.zeros((self.env.get_state_size(), self.env.get_action_size()))

    # Solve the environment using Q-learning algorithm
    def solve(self, epsilon=0.1, tau=1.0, seed=None):
        np.random.seed(seed)
        total_rewards = []
        total_steps = []

        for episode in range(self.episodes):
            state = self.env.reset()[1]
            done = False
            total_reward = 0
            steps = 0

            while not done:
                action = self.select_action(self.q_table, state, epsilon, tau)
                _, next_state, reward, done = self.env.step(action)

                if not self.env.is_terminal(next_state):
                    next_max = np.max(self.q_table[next_state, :])
                else:
                    next_max = 0

                self.q_table[state, action] += self.alpha * (reward + self.gamma * next_max - self.q_table[state, action])

                state = next_state
                total_reward += reward
                steps += 1

            total_rewards.append(total_reward)
            total_steps.append(steps)

        policy = np.argmax(self.q_table, axis=1)
        return policy, total_rewards, total_steps

```

Fig. 11. Base Q Learning Agent inheriting RLagent

```

# SARSA agent
class SARSAAgent(RLagent):

    def __init__(self, env, policy_type='softmax', alpha=0.1, gamma=0.99, episodes=1000):
        super().__init__(env, policy_type, alpha, gamma, episodes)
        self.q_table = np.zeros((self.env.get_state_size(), self.env.get_action_size()))

    # Solve the environment using SARSA algorithm
    def solve(self, epsilon=0.1, tau=1.0, seed=None):
        np.random.seed(seed)
        total_rewards = []
        total_steps = []

        for episode in range(self.episodes):
            state = self.env.reset()[1]
            action = self.select_action(self.q_table, state, epsilon, tau)
            done = False
            total_reward = 0
            steps = 0

            while not done:
                _, next_state, reward, done = self.env.step(action)
                next_action = self.select_action(self.q_table, next_state, epsilon, tau)

                if not self.env.is_terminal(next_state):
                    td_target = reward + self.gamma * self.q_table[next_state, next_action]
                else:
                    td_target = reward

                td_error = td_target - self.q_table[state, action]
                self.q_table[state, action] += self.alpha * td_error

                state, action = next_state, next_action
                total_reward += reward
                steps += 1

            total_rewards.append(total_reward)
            total_steps.append(steps)

        policy = np.argmax(self.q_table, axis=1)
        return policy, total_rewards, total_steps

```

Fig. 12. Base SARSA Agent inheriting RLagent

APPENDIX C

CHOOSING BEST HYPERPARAMETERS FOR SINGLE TARGET DETERMINISTIC REWARD FOR ϵ -GREEDY.

Agent	Gamma	Alpha	Epsilon	Avg Reward
SARSA	0.9	0.1	0.8	-8.00
SARSA	0.5	0.5	0.5	-5.00
SARSA	0.1	0.9	0.2	-1.00
SARSA	0.1	0.5	0.5	-1.00
SARSA	0.5	0.5	0.1	350.00
SARSA	0.5	0.5	0.9	-8.00
SARSA	0.8	0.2	0.7	-7.00
SARSA	0.6	0.4	0.4	147.00
SARSA	0.3	0.7	0.3	0.00
SARSA	0.2	0.8	0.6	-4.00
SARSA	0.7	0.3	0.8	-4.00
SARSA	0.4	0.6	0.5	0.00
SARSA	0.5	0.4	0.3	149.00
SARSA	0.2	0.7	0.9	-9.00
SARSA	0.6	0.5	0.2	249.00
SARSA	0.3	0.6	0.4	48.00
SARSA	0.4	0.5	0.6	-1.00
SARSA	0.7	0.4	0.3	298.00
SARSA	0.5	0.7	0.8	-4.00
SARSA	0.8	0.3	0.2	197.00
Q-learning	0.9	0.1	0.8	92.00
Q-learning	0.5	0.5	0.5	41.00
Q-learning	0.1	0.9	0.2	449.00
Q-learning	0.1	0.5	0.5	245.00
Q-learning	0.5	0.5	0.1	347.00
Q-learning	0.5	0.5	0.9	-10.00
Q-learning	0.8	0.2	0.7	92.00
Q-learning	0.6	0.4	0.4	245.00
Q-learning	0.3	0.7	0.3	245.00
Q-learning	0.2	0.8	0.6	143.00
Q-learning	0.7	0.3	0.8	-10.00
Q-learning	0.4	0.6	0.5	143.00
Q-learning	0.5	0.4	0.3	143.00
Q-learning	0.2	0.7	0.9	41.00
Q-learning	0.6	0.5	0.2	296.00
Q-learning	0.3	0.6	0.4	92.00
Q-learning	0.4	0.5	0.6	-10.00
Q-learning	0.7	0.4	0.3	245.00
Q-learning	0.5	0.7	0.8	41.00
Q-learning	0.8	0.3	0.2	296.00

TABLE I

RANDOM SEARCH RESULTS FOR SARSA AND Q-LEARNING FOR ϵ -GREEDY

APPENDIX D

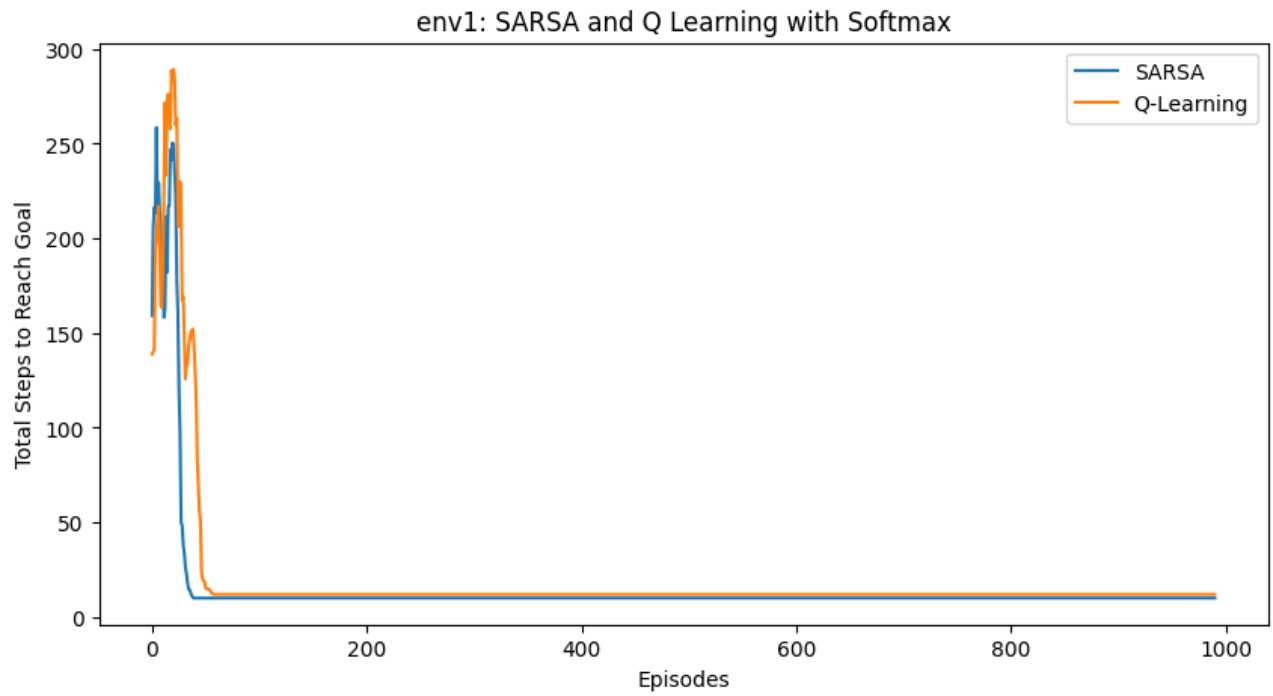
IMPLEMENTING SOFTMAX

This code defines a 'softmax' function that normalizes the Q-values using the softmax technique, which transforms them into probabilities. The temperature parameter 'tau' controls the randomness of the selection, where lower values make the selection more greedy (favoring higher Q-values) and higher values make the selection more random.

```
def softmax(self, q_values, tau=1.0):  
    exp_q = np.exp(q_values / tau - np.max(q_values / tau))  
    return exp_q / np.sum(exp_q)
```

Fig. 13. Implementing softmax for RL Agents

APPENDIX E
COMPARING NUMBER OF STEPS TAKEN BY Q LEARNING AND SARSA AGENT IN ENVIRONMENT 1 AND ENVIRONMENT 2



APPENDIX F DECAYING TAU

This code snippet updates the value of 'tau' by decaying it over time, ensuring that it does not fall below a predefined minimum value ('self.tau_min'). The decay is controlled by 'self.decay_rate', which reduces 'tau' gradually to balance exploration and exploitation during training.

```
self.tau = max(self.tau_min, self.tau * self.decay_rate)
```

Fig. 14. Decaying Tau inside solve functions

APPENDIX G IMPLEMENTING ELIGIBILITY TRACES

The 'QLearningSoftmaxDecayingTauTraces' and 'SARSA SoftmaxDecayingTauTraces' classes implement an advanced Q-learning algorithm that incorporates softmax action selection with a decaying 'tau' parameter and eligibility traces. In the 'solve' method, for each episode, the environment is reset, and eligibility traces are initialized. Actions are selected using the softmax policy with the current 'tau', and Q-values are updated based on the reward and observed state transitions, utilizing eligibility traces for more efficient learning. The 'tau' parameter decays over time to balance exploration and exploitation. The method tracks visits and cumulative rewards for goal locations and returns the Q-table, goal visits, cumulative rewards, and total steps taken. This implementation enhances learning efficiency by integrating multiple advanced RL techniques.


```

# Q-learning with eligibility traces
class QLearningSoftmaxDecayingTauTraces(QLearningSoftmaxDecayingTau):
    def __init__(self, env, tau_initial=1.0, tau_min=0.1, decay_rate=0.99, alpha=0.1, gamma=0.9, lambda_trace=0.8, episodes=100):
        super().__init__(env, tau_initial, tau_min, decay_rate, alpha, gamma, episodes)
        self.lambda_trace = lambda_trace
        self.goal_1_visits = []
        self.goal_2_visits = []
        self.goal_1_rewards = []
        self.goal_2_rewards = []

    # Solve the environment with eligibility traces
    def solve(self, seed=None):
        np.random.seed(seed)
        self.e_table = np.zeros_like(self.q_table)
        goal_visits = {goal: 0 for goal in self.env.get_goal_loc()}
        cumulative_rewards = {goal: 0 for goal in self.env.get_goal_loc()}

        for episode in range(self.episodes):
            t, state, reward, done = self.env.reset()
            total_reward = 0
            steps = 0
            self.e_table.fill(0) # Reset eligibility traces

            while not done:
                action = self.select_action(self.q_table, state, tau=self.tau)
                t, next_state, reward, done = self.env.step(action)
                total_reward += reward
                steps += 1

                best_next_action = np.argmax(self.q_table[next_state])
                td_target = reward + self.gamma * self.q_table[next_state, best_next_action]
                td_error = td_target - self.q_table[state, action]

                self.e_table *= self.gamma * self.lambda_trace
                self.e_table[state, action] += 1
                self.q_table += self.alpha * td_error * self.e_table

                state = next_state

            self.total_steps.append(steps)
            self.tau = max(self.tau_min, self.tau * self.decay_rate)

            for goal in self.env.get_goal_loc():
                goal_state = self.env._get_state_from_loc(goal)
                if state == goal_state:
                    goal_visits[goal] += 1
                    cumulative_rewards[goal] += total_reward

            self.goal_1_visits.append(goal_visits[self.env.get_goal_loc()[0]])
            self.goal_2_visits.append(goal_visits[self.env.get_goal_loc()[1]])
            self.goal_1_rewards.append(cumulative_rewards[self.env.get_goal_loc()[0]])
            self.goal_2_rewards.append(cumulative_rewards[self.env.get_goal_loc()[1]])

        return self.q_table, goal_visits, cumulative_rewards, self.total_steps

```

Fig. 15. Modified Q Learning Agent with Eligibility Traces

```

# SARSA with eligibility traces
class SARSASoftmaxDecayingTauTraces(SARSASoftmaxDecayingTau):
    def __init__(self, env, tau_initial=1.0, tau_min=0.1, decay_rate=0.99, alpha=0.1, gamma=0.9, lambda_trace=0.8, episodes=100):
        super().__init__(env, tau_initial, tau_min, decay_rate, alpha, gamma, episodes)
        self.lambda_trace = lambda_trace
        self.goal_1_visits = []
        self.goal_2_visits = []
        self.goal_1_rewards = []
        self.goal_2_rewards = []

# Solve the environment with eligibility traces
def solve(self, seed=None):
    np.random.seed(seed)
    self.e_table = np.zeros_like(self.q_table)
    goal_visits = {goal: 0 for goal in self.env.get_goal_loc()}
    cumulative_rewards = {goal: 0 for goal in self.env.get_goal_loc()}

    for episode in range(self.episodes):
        t, state, reward, done = self.env.reset()
        total_reward = 0
        steps = 0
        self.e_table.fill(0) # Reset eligibility traces

        action = self.select_action(self.q_table, state, tau=self.tau)

        while not done:
            t, next_state, reward, done = self.env.step(action)
            total_reward += reward
            steps += 1
            next_action = self.select_action(self.q_table, next_state, tau=self.tau)

            td_target = reward + self.gamma * self.q_table[next_state, next_action]
            td_error = td_target - self.q_table[state, action]

            self.e_table *= self.gamma * self.lambda_trace
            self.e_table[state, action] += 1
            self.q_table += self.alpha * td_error * self.e_table

            state, action = next_state, next_action

        self.total_steps.append(steps)
        self.tau = max(self.tau_min, self.tau * self.decay_rate)

        for goal in self.env.get_goal_loc():
            goal_state = self.env._get_state_from_loc(goal)
            if state == goal_state:
                goal_visits[goal] += 1
                cumulative_rewards[goal] += total_reward

        self.goal_1_visits.append(goal_visits[self.env.get_goal_loc()[0]])
        self.goal_2_visits.append(goal_visits[self.env.get_goal_loc()[1]])
        self.goal_1_rewards.append(cumulative_rewards[self.env.get_goal_loc()[0]])
        self.goal_2_rewards.append(cumulative_rewards[self.env.get_goal_loc()[1]])

    return self.q_table, goal_visits, cumulative_rewards, self.total_steps

```

Fig. 16. Modified SARSA Agent with Eligibility Traces

APPENDIX H

IMPLEMENTING SIMPLER ANN

The ‘SimpleNeuralNetworkReLU’ class initializes a neural network with ReLU activation, random weights, and biases. It includes methods to predict outputs (‘predict’), apply the ReLU function (‘relu’), and update weights and biases based on the error between predictions and targets (‘update’). The learning rate controls the update magnitude.

```
class SimpleNeuralNetworkReLU:
    def __init__(self, input_size, output_size, learning_rate=0.01):
        self.input_size = input_size
        self.output_size = output_size
        self.learning_rate = learning_rate

        # Initialize weights using standard normal distribution
        self.weights = np.random.randn(input_size, output_size) * np.sqrt(2. / input_size)
        self.bias = np.zeros((1, output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def predict(self, state):
        linear_output = np.dot(state, self.weights) + self.bias
        return self.relu(linear_output)

    def update(self, state, target):
        output = self.predict(state)
        error = target - output

        # Directly update weights and biases
        self.weights += self.learning_rate * np.dot(state.T, error)
        self.bias += self.learning_rate * error
```

```

def solve(self, seed=None):
    np.random.seed(seed)
    goal_visits = {goal: 0 for goal in self.env.get_goal_loc()}
    cumulative_rewards = {goal: 0 for goal in self.env.get_goal_loc()}

    for episode in range(self.episodes):
        t, state, reward, done = self.env.reset()
        one_hot_state = np.eye(self.state_size)[state].reshape(1, -1)

        while not done:
            action = self.choose_action(one_hot_state)
            t, next_state, reward, done = self.env.step(action)
            next_one_hot_state = np.eye(self.state_size)[next_state].reshape(1, -1)

            best_next_action = np.argmax(self.q_network.predict(next_one_hot_state))
            td_target = reward + self.gamma * self.q_network.predict(next_one_hot_state)[0][best_next_action]
            target_f = self.q_network.predict(one_hot_state)[0]
            target_f[action] = td_target
            self.q_network.update(one_hot_state, target_f.reshape(1, -1))

            one_hot_state = next_one_hot_state
            state = next_state

        for goal in self.env.get_goal_loc():
            goal_state = self.env._get_state_from_loc(goal)
            if state == goal_state:
                goal_visits[goal] += 1
                cumulative_rewards[goal] += reward

        self.goal_1_visits.append(goal_visits[self.env.get_goal_loc()[0]])
        self.goal_2_visits.append(goal_visits[self.env.get_goal_loc()[1]])
        self.goal_1_rewards.append(cumulative_rewards[self.env.get_goal_loc()[0]])
        self.goal_2_rewards.append(cumulative_rewards[self.env.get_goal_loc()[1]])

        self.tau = max(self.tau_min, self.tau * self.decay_rate)

    return self.q_network, goal_visits, cumulative_rewards, self.total_steps

```

Fig. 17. solve() for Q Learning Agent using ANN

```

def solve(self, seed=None):
    np.random.seed(seed)
    goal_visits = {goal: 0 for goal in self.env.get_goal_loc()}
    cumulative_rewards = {goal: 0 for goal in self.env.get_goal_loc()}

    for episode in range(self.episodes):
        t, state, reward, done = self.env.reset()
        one_hot_state = np.eye(self.state_size)[state].reshape(1, -1)

        action = self.choose_action(one_hot_state)

        while not done:
            t, next_state, reward, done = self.env.step(action)
            next_one_hot_state = np.eye(self.state_size)[next_state].reshape(1, -1)
            next_action = self.choose_action(next_one_hot_state)

            td_target = reward + self.gamma * self.q_network.predict(next_one_hot_state)[0][next_action]
            target_f = self.q_network.predict(one_hot_state)[0]
            target_f[action] = td_target
            self.q_network.update(one_hot_state, target_f.reshape(1, -1))

            one_hot_state = next_one_hot_state
            action = next_action
            state = next_state

        for goal in self.env.get_goal_loc():
            goal_state = self.env._get_state_from_loc(goal)
            if state == goal_state:
                goal_visits[goal] += 1
                cumulative_rewards[goal] += reward

        self.goal_1_visits.append(goal_visits[self.env.get_goal_loc()[0]])
        self.goal_2_visits.append(goal_visits[self.env.get_goal_loc()[1]])
        self.goal_1_rewards.append(cumulative_rewards[self.env.get_goal_loc()[0]])
        self.goal_2_rewards.append(cumulative_rewards[self.env.get_goal_loc()[1]])

        self.tau = max(self.tau_min, self.tau * self.decay_rate)

    return self.q_network, goal_visits, cumulative_rewards, self.total_steps

```

Fig. 18. solve() for SARSA Agent using ANN

APPENDIX I

ANN WITH PROPAGATION AND GRADIENT DESCENT

```
# Simple neural network for approximating Q-functions
class SimpleNeuralNetwork:
    """A simple neural network for approximating Q-functions with one hidden layer and ReLU activation."""
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate

        # Initialize weights with a more stable method (He initialization for ReLU)
        self.weights_input_to_hidden = np.random.randn(input_size, hidden_size) * np.sqrt(2. / input_size)
        self.weights_hidden_to_output = np.random.randn(hidden_size, output_size) * np.sqrt(2. / hidden_size)

    def relu(self, x):
        """ReLU activation function."""
        return np.maximum(0, x)

    def relu_derivative(self, x):
        """Derivative of ReLU activation function."""
        return (x > 0).astype(float)

    def predict(self, state):
        """Forward pass through the network using ReLU."""
        self.hidden_layer = self.relu(np.dot(state, self.weights_input_to_hidden))
        return np.dot(self.hidden_layer, self.weights_hidden_to_output)

    def update(self, state, target_f):
        """Backward pass with gradient descent update using ReLU."""
        predicted_f = self.predict(state)
        output_error = target_f - predicted_f
        hidden_error = np.dot(output_error, self.weights_hidden_to_output.T) * self.relu_derivative(self.hidden_layer)

        # Gradient clipping
        clip_value = 1 # You can adjust this threshold based on your specific needs
        np.clip(hidden_error, -clip_value, clip_value, out=hidden_error)

        # Gradient descent
        self.weights_hidden_to_output += self.learning_rate * np.dot(self.hidden_layer.T, output_error)
        self.weights_input_to_hidden += self.learning_rate * np.dot(state.T, hidden_error)

        # Debug information
        if np.any(np.isnan(self.weights_input_to_hidden)) or np.any(np.isnan(self.weights_hidden_to_output)):
            print("NaN detected in weights after update")
```